CAIO ACADEMY

OPERATIONS

CAIO Academy Infrastructure Architecture

Dedicated VPS, multi-project Convex, Next.js dashboards per C-Level, email allowlist, multi-role orchestration — the full technical architecture

AUTEUR CAIO Academy

DATE 4 mai 2026

ÉDITION 2026 Edition

Sommaire

Infrastructure & technical architecture – CAIO Academy

Overview

The dedicated VPS

AI authentication

Multi-project Convex backend

Next.js dashboards

Email allowlist

Multi-role orchestration

Integrations & automations

Stack summary

Deployment flow

Why this architecture

© 2026 CAIO Academy

Infrastructure & technical architecture – CAIO Academy

> *Source of truth on how the CAIO Academy team builds, deploys, and operates its C-Level dashboards. The architecture is deliberately centralized: one dedicated VPS, one parent Convex account, N Convex projects (one per C-Level), N Next.js dashboards on Vercel, all gated by an email allowlist. All development and orchestration work happens on this single machine, never on local laptops.*

Overview

The architecture stacks five layers: access, compute, AI, backend, frontend. Each layer has a single responsibility and a single entry point, which makes the system inspectable, reproducible, and auditable.

© 2026 CAIO Academy

```
+-------------------------------------------------------------+
|                        Team members                         |
|              (developers + orchestrators)                   |
+----------------------+--------------------------------------+
                       | SSH (port 42820, key-only)
                       v
+-------------------------------------------------------------+
|                        Dedicated VPS                        |
|   single user : hacker          shell : zsh                 |
|   tmux multi-session  |  git  |  Node 20  |  Bun            |
|   Convex CLI  |  Vercel CLI  |  Claude Code CLI             |
+----------------------+--------------------------------------+
                       |
                       v
+-------------------------------------------------------------+
|              Claude Code credential pool                    |
|   N Claude Code Max accounts (flat-rate, no per-token) +    |
|   ANTHROPIC_API_KEY (fallback for headless / non-Max tools) |
|   commands : /switch, /switch-add                           |
+----------------------+--------------------------------------+
                       |
                       v
+-------------------------------------------------------------+
|         Parent Convex account (1 organization)              |
|  +---------+---------+---------+---------+---------+        |
|  |  CTO    |  CMO    |  CPO    |  CFO    |  CHRO   |  ...   |
|  | project | project | project | project | project |        |
|  +----+----+----+----+----+----+----+----+----+----+        |
+-------|---------|---------|---------|---------|-------------+
        |         |         |         |         |
        v         v         v         v         v
   +--------+ +--------+ +--------+ +--------+ +--------+
   |  CTO   | |  CMO   | |  CPO   | |  CFO   | | CHRO   |
   |  dash  | |  dash  | |  dash  | |  dash  | |  dash  |
   |Next.js | |Next.js | |Next.js | |Next.js | |Next.js |
   +---+----+ +---+----+ +---+----+ +---+----+ +---+----+
       |          |          |          |          |
       +----------+----+-----+----------+----------+
                       v
              +-----------------------------------+
              | Vercel (1 deploy per dashboard)   |
              | Domains : <role>.caio-academy.x   |
              +-------+---------------------------+
                      |
                      v
              +----------------------------+
              | Email allowlist gate       |
```


© 2026 CAIO Academy

```
                | (1 list per C-Level dash)   |
                +-----------------------------+
```

The guiding principle: one C-Level = one Convex project = one Next.js dashboard = one allowlist. This 1-1-1-1 mapping eliminates cross-coupling and simplifies permissions, quotas, and incident handling.

The dedicated VPS

The heart of the system is an Ubuntu VPS hosted with a European provider, reachable only via SSH on a non-standard port with key-based authentication. All code, all orchestration, all tmux sessions live here. No developer pushes from a personal laptop.

Why centralize

- **Reproducibility**: one environment, one Node version, one Bun, one configured Vercel CLI. No “works on my machine”.
- **Security**: Convex and Vercel credentials never leave the VPS. A lost laptop exposes nothing.
- **Continuity**: tmux preserves running sessions across disconnects. A 40-minute build keeps going even if the operator closes their laptop.
- **Observability**: every log, every process, every cron job lives in one place, readable by the whole team.

Team access

Each team member adds their own SSH key to the `hacker` account. Key rotation is manual: the key is removed from `~/.ssh/authorized_keys` when a member leaves the project.

```
# Standard connection
ssh -p 42820 hacker@<vps-ip>

# Attach to an existing tmux session (e.g. CTO dashboard dev)
tmux attach -t CAIO-CTO-dev

# List sessions
tmux ls
```

tmux conventions

Sessions follow the pattern `CAIO-<role>-<type>`:

- `CAIO-CTO-dev`: daily dev for the CTO dashboard
- `CAIO-CMO-fix-auth`: dispatched worker fixing CMO dashboard auth
- `CAIO-oracle-1`: orchestration Oracle (see orchestration section)

© 2026 CAIO Academy

The naming convention lets any operator recognize the context of a session at a glance, without having to ask.

AI authentication

CAIO Academy uses Claude Code heavily, both for development and for orchestration. Two authentication modes coexist.

Claude Code Max accounts (primary mode)

Several Claude Code Max accounts are pooled together. Each account carries an effectively unlimited token budget within fair-use limits, which avoids per-token API billing entirely. For sessions running multiple hours per day, the savings are roughly 90 to 95 percent compared to equivalent API consumption.

The pool is managed by two simple commands:

```
# Add a new account to the pool
/switch-add

# Rotate to the next account
/switch claude-account-2
```

Manual rotation prevents single-account rate limiting during heavy workloads (Quality Arsenal audits, bulk content generation). Each account is identified by a short name and isolated in its own credentials directory.

Anthropic API key (fallback)

For tools that don't support Claude Code authentication (batch scripts, CI/CD integrations, headless agents), an `ANTHROPIC_API_KEY` is exported from `~/.zshrc.private`. This key bills per usage and is reserved for automation paths that can't use the Max pool.

When to use which

Use case	Auth	Reason
Daily interactive dev	Claude Code Max	Flat-rate, no billing surprises
Quality Arsenal audits (16 protocols)	Claude Code Max	High volume absorbed by flat-rate
Convex cron generating a report	API key	Headless, no interactive session
Reactive webhook	API key	No TTY, automated batch

© 2026 CAIO Academy

Multi-project Convex backend

Convex is the backend source of truth. The notable choice: we do not use a single Convex project for everything. Instead, **one parent account** (Convex organization) holds **N projects**, one per C-Level.

Why one project per C-Level

- **Data isolation**: data in the CTO dashboard (incidents, deploys, repos) has no business being readable from the CMO dashboard (campaigns, leads, GA4). Project-level separation makes isolation native, not bolted on.
- **Independent quotas**: each project owns its function quota, bandwidth, and storage. A CMO traffic spike never starves the CTO dashboard.
- **Independent deploys**: a broken schema migration on the CFO project does not break the CHRO project. Each C-Level iterates on its own cadence.
- **Separate API keys**: each Next.js dashboard gets its own `NEXT_PUBLIC_CONVEX_URL` and `CONVEX_DEPLOY_KEY`, simplifying rotation and audit.

Organization by role

C-Level	Convex project	Primary data
CTO	`caio-cto`	Repos, deploys, incidents, infra
CMO	`caio-cmo`	Campaigns, leads, GA4, ad spend
CPO	`caio-cpo`	Products, releases, NPS, churn
CFO	`caio-cfo`	Budgets, MRR, runway, invoices
CHRO	`caio-chro`	Headcount, hires, payroll

Adding a future C-Level (CISO, COO, etc.) follows the same convention: new Convex project, new Next.js dashboard, new allowlist.

Convex capabilities used

Convex covers what would normally be split across four or five external services:

- **Declarative schema** in `convex/schema.ts`, with auto-generated TypeScript types
- **Queries**: reactive reads that re-execute when underlying data changes
- **Mutations**: transactional server-side writes
- **Actions**: external calls (HTTP, AI, Stripe, etc.) without holding mutation locks
- **Cron**: scheduled jobs (`convex/crons.ts`) executed by Convex itself, no external scheduler

© 2026 CAIO Academy

- **Webhooks**: `httpAction` endpoints receiving events from Stripe, Clerk, GitHub
- **Real-time**: native WebSocket subscriptions, no Pusher / Ably to operate

```
// Schema example (illustrative)
export default defineSchema({
  incidents: defineTable({
    severity: v.union(v.literal("p1"), v.literal("p2"), v.literal("p3")),
    title: v.string(),
    openedAt: v.number(),
    closedAt: v.optional(v.number()),
  }).index("by_severity", ["severity"]),
});
```

Next.js dashboards

Every C-Level has a dedicated Next.js dashboard, deployed on Vercel. The source lives in a sub-folder of the CAIO Academy repo (`apps/dashboard-cto`, `apps/dashboard-cmo`, etc.) and each app has its own `next.config.ts`, its own production `package.json`, and its own Vercel project.

Frontend stack

- **Next.js 16** (App Router, Cache Components, Turbopack)
- **React 19** with Server Components
- **Tailwind v4** + shadcn/ui for the design system
- **Clerk** for authentication (see allowlist)
- **convex/react** for real-time backend connectivity
- **Vercel Analytics** + **Vercel Blob** for metrics and active storage

Why one dashboard per role

Each C-Level has distinct visualization needs. The CFO dashboard surfaces financials (MRR, runway, cohort revenue) with dense tables and CSV exports. The CMO dashboard renders funnels, heatmaps, and creative review. Cramming both into a single app produces an unreadable feature-flag soup and prohibitive bundle sizes. One dashboard per role yields a clean bundle, an independent release cycle, and a UI tuned to the decisions of that role.

© 2026 CAIO Academy

Sample environment variables

```
# .env.local for the CTO dashboard
NEXT_PUBLIC_CONVEX_URL=https://caio-cto.convex.cloud
CONVEX_DEPLOY_KEY=prod:...
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_live_...
CLERK_SECRET_KEY=sk_live_...
ALLOWLIST_TEAM=cto
```

Email allowlist

Every dashboard is closed by default. Only emails listed in the allowlist for the matching C-Level can sign in. Any attempt from a non-listed email is rejected at the auth layer, before any data is loaded.

Mechanism

Three defense layers, in order:

1. **Clerk**: the list of authorized emails is configured at the Clerk application level (restricted-access mode). No public signup, no self-serve onboarding.
2. **Next.js middleware**: every request verifies that the email in the Clerk JWT belongs to the dashboard's allowlist, otherwise redirects to `/forbidden`.
3. **Convex**: every query and mutation re-checks identity server-side via `ctx.auth.getUserIdentity()` and matches it against the C-Level list. A stolen API key does not bypass this check.

Lists per role

Dashboard	Allowlist	Owner
CTO	`team-cto.json`	maintained by the CTO
CMO	`team-cmo.json`	maintained by the CMO
CPO	`team-cpo.json`	maintained by the CPO
CFO	`team-cfo.json`	maintained by the CFO
CHRO	`team-chro.json`	maintained by the CHRO

The lists are versioned in the repo. Any change goes through a PR, which guarantees auditability: the team always knows when an email was added or removed, and by whom.

© 2026 CAIO Academy

Why no SSO yet

A full SSO setup (Okta, Google Workspace) is on the Year-2 roadmap, once the team grows past 30 people. In Phase I, the static allowlist is sufficient: low volume, rare rotation, simple Git-diff audits.

Multi-role orchestration

Development and operations rely on an Oracle / workers pattern inherited from the broader Agentik OS ecosystem and adapted to CAIO Academy.

Roles

- **Oracle**: long-lived tmux session per project (`CAIO-Academy-oracle-1`). Receives missions, classifies them (SIMPLE / MEDIUM / COMPLEX / EPIC), plans, dispatches.
- **Worker**: ephemeral tmux session (`CAIO-Academy-worker-4-1`). Executes a bounded mission, writes `~/.aisb/state/worker-<session>.done.json` when finished.
- **Patrol**: daemon that reads each worker's `progress.json` and notifies the Oracle / user when a worker stalls.
- **Verifier**: a special worker (Guardian) that validates the build, screenshots, and file scope before closeout.

Session conventions

Pattern	Role	Lifetime
`CAIO-Academy-oracle-N`	Persistent Oracle	Days / weeks
`CAIO-<role>-dev`	Interactive dev on a dashboard	Hours
`CAIO-Academy-worker-N-M`	Dispatched worker	Minutes / hours
`CAIO-Academy-fix-<scope>`	Targeted fix	Minutes

Mission lifecycle

1. A team member opens or attaches to an Oracle session.
2. The Oracle classifies the mission. If COMPLEX, it invokes `/team` , which spawns N teammates as tmux panes.
3. Each worker executes its slice, updates `progress.json` , and writes `done.json` at the end.
4. The Verifier validates the build, the artifacts, and the file scope.
5. The Oracle closes the session (graceful shutdown via `TeamDelete`) and reports back to the team member.

© 2026 CAIO Academy

No worker edits production directly. Push, merge, and deploy are explicit steps the Oracle triggers after verification.

Integrations & automations

Anything event-driven or scheduled runs inside Convex, not in an external service. This keeps the stack cohesive and inspectable from a single place.

Convex cron

Recurring jobs are declared in `convex/crons.ts`:

```
import { cronJobs } from "convex/server";
import { internal } from "./_generated/api";

const crons = cronJobs();

crons.weekly(
  "metrics digest",
  { dayOfWeek: "friday", hourUTC: 15, minuteUTC: 0 },
  internal.reports.weeklyDigest,
);

export default crons;
```

Concrete examples: weekly KPI digest emailed to the team, nightly recompute of retention cohorts, six-hour refresh of LinkedIn / Beehiiv stats.

Inbound webhooks

Convex `httpAction` endpoints serve as HTTPS receivers for Stripe, Clerk, GitHub, Vercel, and ElevenLabs webhooks. Each endpoint verifies the sender signature before any write and logs the raw event into a `webhook_events` table for replay.

Real-time

Dashboards consume Convex queries via `useQuery`. When a mutation updates a row, every open UI re-renders in real time without any custom WebSocket layer. This is the feature that makes Convex strategic for executive dashboards: the CTO sees a new incident appear the moment it is created, with no manual refresh.

© 2026 CAIO Academy

AI integrations

Claude / OpenAI / ElevenLabs agents are called from Convex `action` handlers. Actions are non-transactional, so they can hit external HTTP without holding database locks. A mutation kicks off the action, the action collects the AI response, then a follow-up mutation persists the result.

Stack summary

Layer	Tool	Role	Notes
Compute	Dedicated VPS	Dev + orchestration	SSH only, custom port
Sessions	tmux	Persistent shells	`CAIO-<role>-<type>` naming
AI	Claude Code Max + API	Pooled AI auth	`/switch` between accounts
Backend	Convex (1 parent, N projects)	DB + functions + cron + webhooks	1 project per C-Level
Auth	Clerk + allowlist	Strict email gating	Lists versioned in Git
Frontend	Next.js 16 (App Router)	One dashboard per C-Level	Tailwind + shadcn/ui
Hosting	Vercel	Edge deploys	1 Vercel project per dashboard
Observability	Vercel Analytics + Convex logs	Metrics + traces	Native centralization

---

Deployment flow

The pipeline is unidirectional: development on the VPS, push to GitHub, automatic deploy to Vercel and Convex.

© 2026 CAIO Academy

Step	Actor	Action	Target
1	Developer (worker)	Edits in `apps/dashboard-<role>/`	Local repo on VPS
2	Verifier	`npm run build` + tests	VPS
3	Oracle	Targeted `git add` + `git commit`	Local repo
4	Oracle	`git push`	GitHub
5	Vercel (auto)	Build + Next.js deploy	`https://<role>.caio-academy.x`
6	Convex CLI	`npx convex deploy --prod`	Convex prod for the relevant project
7	Oracle	Smoke check (curl 200, Playwright screenshot)	Production
8	Oracle	Notifies the team (Telegram / email)	–

Step 6 is decoupled from step 5: a pure frontend change does not require a Convex redeploy, and vice versa. This shrinks the blast radius of every release.

Freeze rule

If a deploy fails (red build, failed healthcheck), the project is “frozen”: no further release can ship until the cause is identified. There is no automatic rollback. We deliberately prefer an explicit freeze to a blind rollback that could mask the real root cause.

Why this architecture

Three principles drive the whole setup:

1. **Operational centralization**: a single place to develop and orchestrate, therefore a single place to secure and to monitor.
2. **Isolation by role**: one Convex project and one Next.js dashboard per C-Level eliminate cross-coupling. Each team moves at its own pace without breaking the others.
3. **Auditability**: Git-versioned allowlists, named tmux sessions, declarative cron jobs, logged webhooks. Everything is traceable, replayable, reversible.

The explicit trade-off: we accept the complexity of running N Convex projects and N dashboards instead of a monolith because the separation pays off on security, per-team velocity, and operational legibility.

© 2026 CAIO Academy

This architecture scales cleanly up to roughly a dozen C-Levels and a hundred team members before requiring SSO and multi-VPS federation.

© 2026 CAIO Academy